



Zurich University of Applied Sciences

Department of Life Sciences and Facility Management

Institute for Computational Life Sciences

TECHNICAL PROJECT REPORT

Interactive Air Quality Map Dashboard

Communication and Collaboration in Environmental Science
Group Assignment, Spring 2026

Authors:

Lejla Beganovic
Florian Klaver
Claudio Wismer

Supervisor:

Patrick Laube

Submitted on

2026-05-31

Source code repository: https://github.com/klaveflo/Airquality_Dashboard

Table of Contents

1	Introduction	3
2	Storytelling Concept and Intended Insights	4
3	Key Dashboard Functionalities	5
3.1	Historic Tab	5
3.2	Live Tab	6
4	Setup and Configuration	7
4.1	Required Packages	7
4.2	Load Libraries and Define Global Configuration	7
4.3	European Air Quality Index (EAQI) Configuration	8
5	Datasets	9
5.1	Historic Database (DuckDB)	9
5.1.1	Inspecting the Database Schema	9
5.1.2	Station Metadata	10
5.2	Live Data (EEA Parquet API)	10
5.3	Data Quality	11
6	Historic Tab	12
6.1	Query Functions	12
6.2	Animation Engine	12
6.3	Map Rendering: deck.gl + MapLibre GL	13
6.4	Chart Builders	14
6.5	AI-Powered Peak Explanation	14
7	Live Tab	16
7.1	Live Station Comparison	16
7.2	Application Entry Point	16
8	Discussion and Conclusion	18
8.1	Key Findings	18
8.2	Limitations	18
8.3	Conclusion	18
9	Declaration of Individual Contributions	19
10	Declaration of AI Use	20
	References	21

1 Introduction

Air quality is one of the most pressing environmental and public health challenges in Europe. Long-term exposure to particulate matter (PM_{2.5} and PM₁₀) is responsible for millions of premature deaths globally each year ([World Health Organization, 2021](#)), and the European Environment Agency (EEA) identifies air pollution as the continent’s largest environmental health risk. Concentrations of PM₁₀, PM_{2.5}, NO₂, and O₃ vary substantially across geography and season, driven by industrial emissions, residential heating, road traffic, and meteorology such as temperature inversions and Saharan dust transport. Making this complexity legible through interactive visualisation is essential for both scientific communication and evidence-based policy monitoring.

This project develops an **interactive, map-centred dashboard** built on open data from the **EEA Air Quality e-Reporting (AQeR)** programme ([European Environment Agency, 2024a](#)), the pan-European monitoring network covering more than 40 countries and thousands of stations. It was developed for the ZHAW module *Communication and Collaboration in Environmental Science* (CCES FS26), applying modern communication techniques such as interactive visualisation, animated playback, and AI-assisted interpretation to a real-world environmental dataset. The source code is maintained at https://github.com/klaveflo/Airquality_Dashboard and the dashboard launches with a single terminal command (see Section 4).

The central question guiding the design is:

How have PM₁₀ and PM_{2.5} concentrations evolved across Europe between 2013 and 2024, and what does the current (live) situation look like at station level?

The dashboard answers this through two views: a **Historic tab** for long-term trend exploration with animated playback across more than a decade of daily measurements, and a **Live tab** for near-real-time monitoring of the most recent seven days at hourly resolution.

Extension topic: Animation. The Historic tab implements a Play/Stop animation engine that steps through daily snapshots of the entire monitoring network, rendering colour-coded station dots on a WebGL map via deck.gl ([vis.gl / OpenJS Foundation, 2024](#)). A key constraint was that the map must not reset its zoom or pan state between frames. This was solved by pushing data updates through deck.gl’s `setProps()` API rather than re-mounting the map. The result is a fluid temporal narrative of how pollution evolves through seasons and years, offering insight no static snapshot can.

2 Storytelling Concept and Intended Insights

Effective data visualisation is not only about technical accuracy but about guiding an audience toward understanding through deliberate narrative structure. The dashboard’s design was shaped by three tensions in the data, each motivating a specific design decision.

Tension 1: Then versus now. The split between Historic and Live tabs invites users to connect long-term trends with the present. A user can observe Poland’s gradual PM10 improvement over the decade, then switch to the Live tab to see what Polish stations report today. This past-versus-present device emphasises that historical trends have a direct counterpart in the current world.

Tension 2: East versus West. Embedding the data on a real map makes the persistent disparity between Eastern and Western Europe immediately visible. Poland, Bulgaria, and Romania show systematically higher PM concentrations, reflecting continued reliance on coal and biomass for heating ([European Commission, 2016](#)). This inequality would take paragraphs to convey in prose, but it is grasped in seconds on the map. It carries a message directly relevant to EU policy: despite continent-wide improvement, geographic equity in air quality remains unresolved.

Tension 3: Urban versus rural. Station dots are differentiated by area type. Urban stations use a solid circle, suburban stations use a border ring, and rural stations use a hollow ring. This makes the urban to rural gradient visible at every zoom level without filtering. Research consistently shows urban populations face higher pollutant exposures than rural ones ([World Health Organization, 2021](#)); the map makes this tangible at station level.

The **intended audience** is environmental scientists, policy analysts, and informed members of the public interested in European air quality. The dashboard assumes no prior knowledge of the EEA network but expects basic familiarity with interactive maps and time series. A full exploration session should yield these insights:

1. PM10 and PM2.5 concentrations have declined across most of Europe since 2013, but the pace of improvement is geographically uneven.
2. Winter peaks in particulate concentrations are a structural feature of European air quality, driven primarily by residential heating emissions.
3. Significant country- and city-level disparities persist, with Eastern European stations consistently reporting higher values.
4. Urban monitoring stations record markedly higher pollutant levels than rural stations in the same region.

The **AI-powered peak explanation** feature bridges observed data patterns and real-world events. When a user notices a spike in the daily-average chart, the “Ask AI” button queries the Google Gemini API ([Google DeepMind, 2024](#)) and returns a short explanation. Examples include a Saharan dust intrusion or a temperature-inversion episode. This turns the dashboard from a passive display into an active explanatory environment without the user leaving the application.

3 Key Dashboard Functionalities

The dashboard is organised into two tabs, each targeting a distinct analytical question.

3.1 Historic Tab

Animated map playback. A Play/Stop control steps through daily snapshots of all stations between the selected start and end dates. A five-position slider sets speed from 0.05 to 1.5 seconds per frame and the current date is displayed above the map, labelled *Animating* or *Viewing*. Zoom and pan state are preserved across frames via `deck.gl`'s `setProps()` rather than a full remount. This is the core decision enabling smooth, exploratory animation.

EAQI colour coding. Each dot is coloured by the official six-tier European Air Quality Index ([European Environment Agency, 2023](#)) using a colourblind-friendly blue to yellow to red palette. Missing values render as transparent dots rather than omitted, preserving the network structure and making data gaps explicit.

Area-type differentiation. Urban, suburban, and rural stations are rendered with distinct visual encodings (solid, border-ring, and hollow circles respectively), making the urban to rural gradient legible at any zoom level without requiring explicit filtering.

Country filter with automatic zoom. A dropdown listing every country present in the historic database restricts the map, charts, and statistics to that country's stations and smoothly flies the map to its bounding box, with all charts updating in the same reactive graph.

Daily average time series with interactive date selection. An Altair chart ([VanderPlas et al., 2024](#)) shows the network wide (or country filtered) daily mean against shaded EAQI background bands. Clicking any point or using the date picker jumps the map to that date, enabling seamless navigation between chart and map without breaking the animation state.

Year over Year comparison chart. Up to five representative years are selected automatically and overlaid on a shared January to December axis, making seasonal structure and interannual change directly comparable at a glance.

Jump to Peak. A dedicated button locates the highest daily average in the selected range and snaps the slider, map, and charts to that date. This provides a fast way to find the episode worth explaining with the AI feature, alongside a live period-statistics panel (today's mean, period mean, period peak).

AI-powered peak explanation. An "Ask AI" button sends the currently displayed date, pollutant, and country to a small LLM helper module that returns a brief contextual explanation of likely causal events, supporting interpretation beyond what the data alone can show.

3.2 Live Tab

Seven-day live station map. EEA Parquet files for the most recent seven days are downloaded concurrently using a 20-thread pool executor, parsed, and rendered as a colour-coded station map, fetched per country and pollutant combination on demand via a Load/Refresh control.

Hourly playback and time scrubber. The same non-blocking animation engine as the Historic tab steps through the hourly snapshots of the seven-day window, with a time slider for manual scrubbing and a per-frame counter shown above the map.

Station comparison panel. Clicking two stations (or picking them from a dropdown) opens a side-by-side hourly comparison. EAQI quality bands are rendered as translucent background rectangles, a normalised bar chart summarises the share of hours in each quality category, and per-station summary cards report the dominant category plus best and worst hours of the week.

Pollutant selector. Users can switch between PM10, PM2.5, NO2, and O3. All colours, thresholds, and data queries update dynamically, using pollutant-specific EAQI breakpoints throughout.

4 Setup and Configuration

This report is the technical write-up of the project. It intentionally contains **selected, annotated code excerpts** only, so that the report explains the main implementation decisions without duplicating the full repository. The complete, runnable dashboard code is available in the public Git repository linked on the title page and is launched from there rather than executed inside this document. The codebase uses Shiny for Python ([Posit PBC, 2024](#)) and is split across five Python modules: `app.py` (entry point and shared styling), `shared.py` (constants, EAQI helpers, data fetching, and DuckDB queries), `hist_tab.py` and `live_tab.py` (the two tab UIs and server logic), and `query_llm.py` (the LLM helper for the peak-explanation feature). Run the dashboard from the project root with:

```
shiny run --reload app.py
```

4.1 Required Packages

The following Python packages are required. Run this block once if not already installed.

The required Python packages are `shiny`, `shinywidgets`, `duckdb`, `pandas`, `altair`, `requests`, `pyarrow`, `google-genai`, and `python-dotenv`. Installation instructions and the complete environment setup are provided in the Git repository.

4.2 Load Libraries and Define Global Configuration

```
from functools import lru_cache
from datetime import datetime, timedelta, timezone
from concurrent.futures import ThreadPoolExecutor

import pandas as pd
import altair as alt
import duckdb
import requests
from shiny import App, ui, reactive

EEA_API_URL = "https://eeadmz1-downloads-api-appservice.azurewebsites.net/ParquetFile/urls"
POLLUTANTS = ["PM10", "PM2.5", "NO2", "O3"]
DB_PATH = "eeaopt.db"
HIST_TABLES = {"PM10": "airquality_5", "PM2.5": "airquality_6001"}
```

4.3 European Air Quality Index (EAQI) Configuration

The dashboard uses the official six-tier **European Air Quality Index** ([European Environment Agency, 2023](#)) as the colour scheme throughout both tabs. The palette follows a diverging blue to yellow to red progression that is both perceptually ordered and broadly colourblind-safe.

```
EAQI_THRESHOLDS = {
    "PM2.5": [(5, "Good"), (15, "Fair"), (50, "Moderate"),
              (90, "Poor"), (140, "Very poor"), (float("inf"), "Extremely poor")],
    "PM10": [(15, "Good"), (45, "Fair"), (120, "Moderate"),
             (195, "Poor"), (270, "Very poor"), (float("inf"), "Extremely poor")],
}

EAQI_COLOURS = {
    "Good": "#4477AA", "Fair": "#77AADD", "Moderate": "#DDCC77",
    "Poor": "#EE7733", "Very poor": "#CC3311", "Extremely poor": "#882255",
}

def get_aqi_label(value, pollutant):
    """Return the EAQI category label for a µg/m³ value."""
    if pd.isna(value) or value < 0:
        return None
    for upper, label in EAQI_THRESHOLDS[pollutant]:
        if value <= upper:
            return label
```


5 Datasets

5.1 Historic Database (DuckDB)

The historic data is stored in a local **DuckDB** database ([DuckDB Foundation, 2024](#)), an in-process analytical database optimised for column-oriented queries. This makes it well suited for aggregating millions of measurements without a separate server process. Populated from EEA AQeR bulk downloads, it contains two tables. The table `airquality_5` contains PM10 daily measurements for 29 European countries from 2013 to 2024. The table `airquality_6001` contains PM2.5 daily measurements for 29 European countries from 2013 to 2024.

Each row holds `Date`, `Value` ($\mu\text{g}/\text{m}^3$), `Country` (ISO 3166-1 alpha-2), `Latitude`, and `Longitude`. The combined dataset spans over 40 million records, making columnar storage and query caching essential for interactive response times.

All historic queries are wrapped with Python's `@lru_cache` using the date string and table name as keys, so replaying an animation frame or revisiting a date needs no extra database I/O. This is detailed further in the Historic Tab section.

```
_db_conn = None

def get_db():
    """Return a cached read-only DuckDB connection."""
    global _db_conn
    if _db_conn is None:
        _db_conn = duckdb.connect(DB_PATH, read_only=True)
    return _db_conn
```

5.1.1 Inspecting the Database Schema

```
-- Representative validation queries used during development
DESCRIBE airquality_5;

SELECT COUNT(*) AS n_rows,
       MIN(DATE("Date")) AS first_date,
       MAX(DATE("Date")) AS last_date,
       COUNT(DISTINCT "Country") AS n_countries
FROM airquality_5
WHERE "Value" IS NOT NULL;
```

5.1.2 Station Metadata

A supplementary CSV (`station_metadata_clean.csv`) links sampling-point IDs to names, coordinates, and **area type** for urban, suburban, and rural stations. This is the basis for the three-symbol map differentiation, prepared in `metadata_prep.ipynb` and following the EEA classification vocabulary ([European Environment Agency, 2024b](#)). About 5% of live station IDs could not be matched; these are excluded from the map but retained in time-series charts where coordinates are not required.

```
@lru_cache(maxsize=1)
def get_station_coords():
    """Load station coordinates and area type once, then reuse them."""
    df = pd.read_csv("station_metadata_clean.csv", low_memory=False)
    df.columns = (df.columns.astype(str)
                  .str.replace("\uffff", "", regex=False).str.strip()
                  .str.lower().str.replace(r"[\s\-\.\./]+", "_", regex=True))
    return df[["key", "lat", "lon", "station_name", "area_type"]]
```

5.2 Live Data (EEA Parquet API)

The Live tab fetches the most recent seven days of hourly measurements from the **EEA AQeR download API** ([European Environment Agency, 2024a](#)). A POST request specifies country, pollutant, and a rolling seven-day window ending at the current UTC time; the API returns one Parquet URL per station, downloaded concurrently via a `ThreadPoolExecutor` with 20 worker threads. Parquet was chosen for its columnar compression and column-selective reading without deserialising the full payload.

```
@lru_cache(maxsize=64)
def get_all_station_data_cached(country_code, pollutant):
    """Fetch, clean, cache, and geo-reference seven days of live station data."""
    urls = get_station_urls(country_code, pollutant)
    coords = get_station_coords()

    with ThreadPoolExecutor(max_workers=20) as ex:
        station_frames = [df for df in ex.map(fetch_one_station_file, urls) if df is not
↪ None]

    df_all = pd.concat(station_frames, ignore_index=True)
    df_all = df_all[df_all["Value"] >= 0].dropna(subset=["Start", "Value"])

    # Coordinates and area type are needed for the map, while the full time series
    # is kept for charts and station comparisons.
    df_meta = summarise_stations(df_all).merge(coords, on="key", how="left")
    return df_meta, df_all
```

5.3 Data Quality

The raw EEA data contains a number of quality issues encountered during development. The table below summarises the most significant issues and the handling strategy applied in both the preprocessing pipeline and the live fetch functions.

Table 1: Data quality issues and handling strategies

Issue	Handling strategy
Missing daily station values	Rendered with zero opacity, not omitted
Negative values (instrument errors)	Filtered: <code>Value >= 0</code>
Mixed timestamp formats (ns int / ISO string)	Unified parser, timezone stripped
Unmatched station coordinates (~5%)	Dropped from map, retained in time series
Duplicate entries per station per day	First occurrence kept after deduplication

6 Historic Tab

6.1 Query Functions

All historic queries run against the local DuckDB database around two principles: expensive full-table scans run once and are cached per table, and per-date queries are cached on the ISO date string plus table name. When the animation replays a rendered date, the result returns from `lru_cache` in microseconds rather than re-running SQL.

```
@lru_cache(maxsize=64)
def hist_get_map_data_cached(date_str, table_name):
    """Return one cached daily mean value per station coordinate."""
    conn = get_db()
    return conn.execute(f"""
        SELECT "Latitude" AS lat, "Longitude" AS lon, AVG("Value") AS "Value"
        FROM {table_name}
        WHERE DATE("Date") = '{date_str}'
            AND "Value" IS NOT NULL
            AND "Latitude" IS NOT NULL AND "Longitude" IS NOT NULL
        GROUP BY "Latitude", "Longitude"
    """).fetch_df()

@lru_cache(maxsize=64)
def hist_get_daily_averages(table_name, start, end, country="ALL"):
    """Return daily averages for the selected period and optional country filter."""
    country_clause = f'AND "Country" = '{country}' if country != "ALL" else ""
    return get_db().execute(f"""
        SELECT DATE("Date") AS "Date", AVG("Value") AS AvgValue
        FROM {table_name}
        WHERE "Value" IS NOT NULL
            AND DATE("Date") BETWEEN '{start}' AND '{end}'
            {country_clause}
        GROUP BY DATE("Date") ORDER BY "Date"
    """).fetch_df()
```

6.2 Animation Engine

The animation engine is the centrepiece of the extension topic. The key decision is to use `reactive.invalidate_later()` rather than a blocking `while` loop, keeping Shiny's event loop responsive so the user can change speed, stop, or interact with any UI element during playback. Each call to `_hist_advance()` schedules its own next execution. This creates a non-blocking timer firing once per frame at the chosen interval.

```
# Speed mapping: slider position 1 to 5 controls seconds per frame
_SPEED_MAP = {1: 1.5, 2: 0.8, 3: 0.4, 4: 0.15, 5: 0.05}

# Excerpt from hist_server() in hist_tab.py:
def hist_server_animation_excerpt(input, output, session):

    hist_playing = reactive.Value(False)
    hist_anim_date = reactive.Value(None)

    @reactive.effect
    @reactive.event(input.hist_play)
    def _hist_play():
        hist_playing.set(True)
        hist_anim_date.set(input.hist_date())

    @reactive.effect
    @reactive.event(input.hist_stop)
    def _hist_stop():
        hist_playing.set(False)

    @reactive.effect
    def _hist_advance():
        if not hist_playing():
            return
        speed = _SPEED_MAP.get(input.hist_speed(), 0.5)
        reactive.invalidate_later(speed)          # re-fires after `speed` seconds
        with reactive.isolate():
            cur = hist_anim_date() if hist_anim_date() is not None else input.hist_date()
            nxt = cur + _dt.timedelta(days=1)
            if nxt > input.hist_end():
                hist_playing.set(False)
            else:
                hist_anim_date.set(nxt)              # triggers all downstream reactives
```

6.3 Map Rendering: deck.gl + MapLibre GL

The map is a static `<div id="deck-hist-map">` mounted once at page load. **deck.gl** ([vis.gl / OpenJS Foundation, 2024](https://vis.gl/)) renders three `ScatterplotLayer` instances for urban, suburban, and rural stations on a **MapLibre GL** ([MapLibre Contributors, 2024](https://maplibre.org/)) base map. Updates arrive via Shiny's WebSocket channel and are applied with `setProps()`, so no re-mount occurs across frames or country changes and zoom/pan state is fully preserved.

```
def build_hist_map_payload(df):
    """Prepare separate layer payloads for urban, suburban, and rural stations."""
    norm = df["area_type"].str.lower().fillna("unknown")
```

```

groups = {
    "urban": df[norm.isin(["urban", "unknown"])],
    "suburban": df[norm == "suburban"],
    "rural": df[norm.isin(["rural", "rural-nearcity", "rural_nearcity"])],
}
return {
    key: value[["lon", "lat", "color_r", "color_g", "color_b", "color_a",
                "value_str", "station_name", "aqi_label", "area_type"]]
        .where(value.notna(), None).to_dict("records")
    for key, value in groups.items()
}

```

6.4 Chart Builders

```

def build_hist_avg_chart(df_avg, current_date):
    """Build the clickable daily-average chart and highlight the animated date."""
    df = df_avg.copy()
    df["DateStr"] = df["Date"].dt.strftime("%Y-%m-%d")
    date_param = alt.param(name="histCurDate", value=str(current_date))
    click_sel = alt.selection_point(name="date_click", on="click", nearest=True,
                                    fields=["DateStr"])

    line = alt.Chart(df).mark_line().encode(x="Date:T", y="AvgValue:Q")
    active_dot = (alt.Chart(df).mark_circle(size=100)
                  .encode(x="Date:T", y="AvgValue:Q")
                  .transform_filter("datum.DateStr === histCurDate"))
    click_layer = (alt.Chart(df).mark_point(opacity=0.001, size=300)
                  .encode(x="Date:T", y="AvgValue:Q").add_params(click_sel))

    return alt.layer(line, active_dot, click_layer).add_params(date_param)

```

6.5 AI-Powered Peak Explanation

The Historic tab's "Ask AI" button calls `ask_llm_about_peak()` in the dedicated `query_llm.py` module, passing the currently selected date, pollutant, and country. The model (`gemini-2.5-flash`, via the **Google Gemini API** ([Google DeepMind, 2024](#))) returns a one to two sentence explanation of the likely cause of an observed peak. Its scope is deliberately narrow. It focuses on environmental context rather than general air-quality information, keeping responses concise and actionable.

```

# query_llm.py
from google import genai

def ask_llm_about_peak(date_str, pollutant, country="ALL"):

```

```
"""Query Google Gemini for a contextual peak explanation."""
if not os.environ.get("GEMINI_API_KEY"):
    return "Error: GEMINI_API_KEY not set."
try:
    client = genai.Client()
    region = "Europe" if country == "ALL" else country
    prompt = (
        f"Explain briefly in only 1-2 sentences what event could cause "
        f"an air quality ({pollutant}) peak on {date_str} in {region}. "
        f"Keep your response short and concise."
    )
    response = client.models.generate_content(
        model="gemini-2.5-flash", contents=prompt)
    return response.text
except Exception as e:
    return f"Error communicating with Gemini API: {e}"
```

7 Live Tab

7.1 Live Station Comparison

Clicking two stations on the live map opens a comparison panel: an hourly time series overlaid with EAQI band backgrounds, a normalised bar chart showing the share of hours in each quality category over the seven days, and per-station summary cards (dominant category, best and worst hour). The bands are built programmatically from pollutant-specific thresholds, so the chart is correct for all four pollutants without hard-coded values.

```
def build_comparison_chart(df_sel, pollutant):
    """Compare two stations against pollutant-specific EAQI background bands."""
    bands = build_eaqi_band_layer(pollutant, df_sel["Value"].max())
    lines = (alt.Chart(df_sel).mark_line(point=True)
             .encode(x="Start:T",
                    y=alt.Y("Value:Q", title=f"{pollutant} (µg/m³)"),
                    color=alt.Color("station_name:N", title="Station"),
                    tooltip=["Start:T", "Value:Q", "station_name:N"]))
    return alt.layer(bands, lines).resolve_scale(color="independent")
```

7.2 Application Entry Point

The two tab modules are assembled in `app.py`, which defines the top-level UI and server, injects the shared CSS, and passes everything to Shiny's App constructor ([Posit PBC, 2024](#)). Shared CSS supports dark and light mode via `@media (prefers-color-scheme: light)`, and CDN-hosted libraries (MapLibre GL, deck.gl) load in the HTML `<head>` to avoid bundling.

```
# app.py entry point
from shiny import App, ui
from live_tab import live_ui, live_server
from hist_tab import hist_ui, hist_server

app_ui = ui.page_fluid(
    ui.head_content(
        ui.tags.link(rel="stylesheet",
                    href="https://unpkg.com/maplibre-gl@4.7.1/dist/maplibre-gl.css"),
        ui.tags.script(
            src="https://unpkg.com/maplibre-gl@4.7.1/dist/maplibre-gl.js"),
        ui.tags.script(
            src="https://unpkg.com/deck.gl@9.0.33/dist.min.js"),
    ),
    ui.h1("Air Quality Map Dashboard"),
    ui.navset_tab(
        live_ui(), # Live Data tab
```



```
        hist_ui(),    # Historic Data tab
        selected="Live Data",
    ),
)

def server(input, output, session):
    live_server(input, output, session)
    hist_server(input, output, session)

app = App(app_ui, server)
```

8 Discussion and Conclusion

8.1 Key Findings

The dashboard makes several patterns from the European air quality literature visually tractable. The animated view immediately reveals that **Eastern European countries**, particularly Poland, Bulgaria, and Romania, show systematically higher PM10 than Western Europe ([European Commission, 2016](#)), consistent with continued reliance on coal and biomass for heating. This disparity persists at the end of the 2013 to 2024 window, suggesting the downward trend has not yet closed the geographic gap.

The Year-over-Year chart summarises the **long-term improvement trend**: most years show a stepwise reduction in peak winter values, consistent with EU emission-reduction directives. The seasonal structure holds across all countries. It includes pronounced winter peaks in December to February and lower summer values, driven across all countries, driven by residential heating combined with temperature-inversion meteorology that traps emissions near the surface.

The area-type differentiation confirms that **urban stations report consistently higher values** than suburban or rural sites in the same region, with direct implications for exposure assessment.

8.2 Limitations

The following limitations should be considered when interpreting the dashboard outputs.

- Daily averaging. Intra-day peaks driven by traffic rush hours or episodic industrial releases are averaged out at the daily granularity stored in the historic database.
- Live API coverage. Not all countries report all four pollutants via the EEA live API, and some country and pollutant combinations return zero station results.
- Coordinate matching. Approximately 5% of live stations cannot be placed on the map due to unresolved sampling-point ID mismatches between the Parquet metadata and the station CSV.
- AI explanations. Gemini-generated peak explanations are contextually plausible but are not verified against official EEA event logs or peer-reviewed post-hoc analyses. They should be treated as indicative interpretations rather than authoritative attribution.

8.3 Conclusion

The dashboard communicates twelve years of European air quality data through animated spatial visualisation, seasonal comparison charts, live monitoring, and AI-assisted interpretation. The animation extension is a compelling narrative tool for exploring how pollution evolves with the seasons and responds to decade scale policy change. This creates an immediacy no static chart

matches. Future work could add weather reanalysis overlays, country-level trend lines, a mobile layout, and automated alerting when live values cross EAQI thresholds.

9 Declaration of Individual Contributions

Each team member confirms that the following table accurately reflects their contributions to the project.

Table 2: Individual contributions by team member

Task	Lejla	Florian	Claudio
Project architecture & Shiny setup		✓	
Historic tab map with deck.gl	✓	✓	
Historic tab animation engine	✓	✓	✓
Historic tab charts with Altair	✓	✓	✓
Historic tab AI integration			✓
Historic tab country filter and auto zoom	✓	✓	✓
Live tab EEA API integration		✓	✓
Live tab map and station comparison	✓	✓	✓
Station metadata preprocessing		✓	
Performance optimisation (lru_cache)	✓	✓	✓
Bug testing & quality assurance	✓	✓	✓
Report writing	✓	✓	✓
Screencast / video	✓	✓	✓

10 Declaration of AI Use

Parts of this implementation were developed with assistance from **Claude** by Anthropic. Specific areas of AI-assisted development include the following items.

- Conversion of the initial Streamlit prototype to Python Shiny.
- Implementation of the deck.gl and MapLibre GL map integration.
- Design of the non-blocking animation engine using `reactive.invalidate_later()`.
- Development of the Vega signal-based chart dot update mechanism that preserves zoom state across animation frames.
- Performance optimisation through `lru_cache` strategies for DuckDB queries.
- Debugging of WebSocket message handling between Python and JavaScript.

All AI-generated code was reviewed, tested, and adapted by the team before integration into the codebase. All scientific and design decisions, including EAQI threshold definitions, chart type selection, storytelling structure, and result interpretation, were made independently by the team.

The “**Ask AI about peaks**” dashboard feature uses the Google Gemini API (`gemini-2.5-flash`) at runtime to generate contextual explanations of pollution peak events. This is an intentional product feature, not a development tool, and is described as such in Section 3 (Key Dashboard Functionalities) and the Historic Tab section.

No AI tools were used for data collection, data preprocessing, statistical analysis, or interpretation of the scientific findings presented in this report.

References

- DuckDB Foundation. (2024). *DuckDB: An in-process SQL OLAP database management system*. DuckDB Foundation. <https://duckdb.org>
- European Commission. (2016). *Directive (EU) 2016/2284 on the reduction of national emissions of certain atmospheric pollutants*. Official Journal of the European Union. <https://eur-lex.europa.eu/eli/dir/2016/2284/oj>
- European Environment Agency. (2023). *European air quality index methodology*. European Environment Agency. <https://www.eea.europa.eu/themes/air/air-quality-index>
- European Environment Agency. (2024a). *Air quality e-reporting (AQeR) parquet download API*. European Environment Agency. <https://eeadmz1-downloads-api-appservice.azurewebsites.net>
- European Environment Agency. (2024b). *Air quality viewer station metadata*. European Environment Agency. <https://discomap.eea.europa.eu/App/AQViewer>
- Google DeepMind. (2024). *Gemini 2.5 flash language model API*. Google DeepMind. <https://ai.google.dev>
- MapLibre Contributors. (2024). *MapLibre GL JS: Open-source map rendering*. MapLibre Contributors. <https://maplibre.org>
- Posit PBC. (2024). *Shiny for python*. Posit PBC. <https://shiny.posit.co/py/>
- VanderPlas, J. et al. (2024). *Altair: Declarative statistical visualization for python*. Altair. <https://altair-viz.github.io>
- vis.gl / OpenJS Foundation. (2024). *Deck.gl: WebGL-powered large-scale data visualization*. OpenJS Foundation. <https://deck.gl>
- World Health Organization. (2021). *WHO global air quality guidelines: Particulate matter (PM_{2.5} and PM₁₀), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide*. World Health Organization. <https://www.who.int/publications/i/item/9789240034228>